Lesson 5: Transformations

Lesson 5: Transformations 1
5.1 2D Transformations 2
5.2 Affine Transformations 2
5.3 Homogeneous Coordinates 3
5.4 Hierarchical Transformations 4
5.5 Transformations in OpenGL 5
Lesson 5 Discussion Session 6

5.1 2D Transformations

Transformation is a process of modifying and re-positioning the existing graphics.

Transformation means changing some graphics into something else by applying rules. We can have various types of transformations such as translation, scaling up or down, rotation, shearing, etc. When a transformation takes place on a 2D plane, it is called 2D transformation.

Transformations play an important role in computer graphics:

1. Change coordinate frames (world, window, viewport, device, etc).
2. Compose objects of simple parts with local scale/position/orientation of one part defined with regard to other parts. For example, for articulated objects.
3. Use deformation to create new shapes.
4. Useful for animation.

There are three basic classes of transformations:

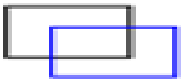
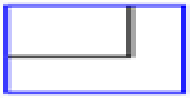
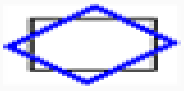
1. **Rigid body** - Preserves distance and angles.
   • Examples: translation and rotation.
2. **Conformal** - Preserves angles.
   • Examples: translation, rotation, and uniform scaling.
3. **Affine** - Preserves parallelism. Lines remain lines.
   • Examples: translation, rotation, scaling, shear, and reflection.

5.2 Affine Transformations

Affine transformation is a linear mapping method that preserves points, straight lines, and planes. Sets of parallel lines remain parallel after an affine transformation.

The affine transformation technique is typically used to correct for **geometric distortions** or **deformations** that occur with non-ideal camera angles. For example, satellite imagery uses affine transformations to correct for wide angle lens distortion, panorama stitching, and image registration. Transforming and fusing the images to a large, flat coordinate system is desirable to eliminate distortion. This enables easier interactions and calculations that don't require accounting for image distortion.

The following table illustrates the different affine transformations: translation, scale, shear, and rotation.

Affine Transform	Example
Translation	
Scale	
Shear	
Rotation	

5.3 Homogeneous Coordinates

To convert a 2×2 matrix to 3×3 matrix, we have to add an extra dummy coordinate W. In this way, we can represent the point by 3 numbers instead of 2 numbers, which is called **Homogenous Coordinate system**. In this system, we can represent all the transformation equations in matrix multiplication.

With homogeneous coordinates, affine transformations become matrices, and composition of transformations is as simple as matrix multiplication.
With homogeneous coordinates, a point $\bar{p}$ is augmented with a 1, to form $\hat{p} = \begin{bmatrix} \bar{p} \\ 1 \end{bmatrix}$.

All points $(\alpha\bar{p}, \alpha)$ represent the same point $\bar{p}$ for real $\alpha \neq 0$.
Given $\hat{p}$ in homogeneous coordinates, to get $\bar{p}$, we divide $\hat{p}$ by its last component and discard the last component.

Example:

The homogeneous points (2, 4, 2) and (1, 2, 1) both represent the Cartesian point (1, 2). It's the orientation of $\hat{p}$ that matters, not its length.
Many transformations become linear in homogeneous coordinates, including affine transformations:

$$\begin{bmatrix} q_x \\ q_y \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} p_x \\ p_y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$
$$= \begin{bmatrix} a & b & t_x \\ c & d & t_y \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix}$$
$$= \begin{bmatrix} A & \vec{t} \end{bmatrix} \hat{p}$$

To produce ˆq rather than ¯q, we can add a row to the matrix:

$$\hat{q} = \begin{bmatrix} A & \vec{t} \\ \vec{0}^T & 1 \end{bmatrix} \hat{p} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \hat{p}.$$

With homogeneous coordinates, the following properties of affine transformations become apparent:
• Affine transformations are associative.
For affine transformations F1, F2, and F3,
(F3 ◦ F2) ◦ F1 = F3 ◦ (F2 ◦ F1).
• Affine transformations are *not* commutative.
For affine transformations F1 and F2,
F2 ◦ F1 ≠ F1 ◦ F2.

The important operations that can be performed correctly in terms of homogeneous coordinates, as follows:
Affine transformations. An important case in the previous section is applying an affine transformation to a point in homogeneous coordinates:
¯q = F(¯p) = A¯p +˜t
qˆ = Aˆpˆ = (x′, y′, 1)T
It is easy to see that this operation is correct, since rescaling ˆp does not change the result:
Aˆ(αpˆ) = α(Aˆpˆ) = αqˆ= (αx′, αy′, α)T
which is the same geometric point as ˆq = (x′, y′, 1)T
Vectors. We can represent a vector ˜v = (x, y) in homogeneous coordinates by setting the last element of the vector to be zero: ˆv = (x, y, 0). However, when adding a vector to a point, the point must have the third component be 1.
ˆq = ˆp + ˆv
(x′, y′, 1)T = (xp, yp, 1) + (x, y, 0)
The result is clearly incorrect if the third component of the vector is not 0.
Note:
Homogeneous coordinates are a representation of points in **projective geometry.**

5.4 Hierarchical Transformations

Hierarchical Transformations is concerned with modeling objects as hierarchically connected parts.

How do we model complex objects and scenes?

- Describing everything as a single complex model is a Bad Idea.
- Use hierarchical modeling instead...

Start with basic set of 3D primitives:
Cubes, spheres, prisms

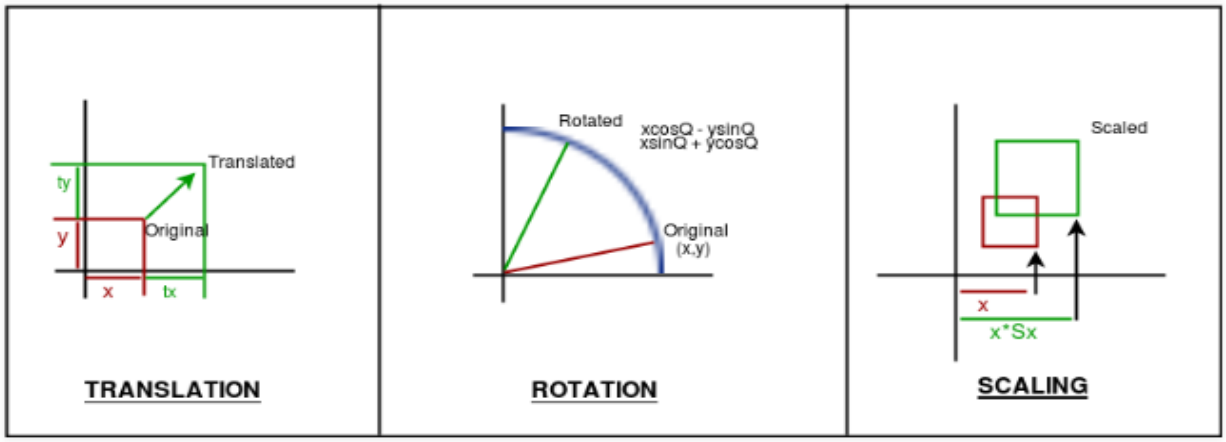
- Each is defined in a ``nice'' way in its own space.
- To put primitives together, use transformations.
- Each transformation embeds one space in another.
- Use a whole hierarchy of spaces to build up complex models...
- Not just one ``Model space'', but one for each model and part of a model.

5.5 Transformations in OpenGL

All transformations are implemented in **2D**.

There are three basic kinds of Transformations in Computer Graphics:

1. Translation
2. Rotation
3. Scaling



1. **Translation:** Translation refers to moving an object to a different position on screen.

   Formula: X = x + tx

   Y = y + ty

   where tx and ty are translation coordinates

   The OpenGL function is **glTranslatef(tx, ty, tz);**

2. **Rotation:** Rotation refers to rotating a point.

   Formula: X = xcosA - ysinA

   Y = xsinA + ycosA,

   A is the angle of rotation.

   The above formula will rotate the point around the origin.

   To rotate around a different point, the formula:

   X = cx + (x-cx)*cosA - (y-cy)*sinA,

   Y = cx + (x-cx)*sinA + (y-cy)*cosA,

cx, cy is centre coordinates,

A is the angle of rotation.

The OpenGL function is **glRotatef (A, x, y, z).**

3. **Scaling:** Scaling refers to zooming in and out an object in different scales across axes.

Formula: X = x*sx

Y = y*sy, sx, sy being scaling factors.

The OpenGL function is **glScalef(float x, float y, float z)**

Note: If combined transformations is to be applied, follow the order: **translate, rotate, scale.**

Lesson 5 Discussion Session

1. Develop C code to implement basic transformations in OPENGL.